Recallit — Flashcard-Based Learning System

Project Overview

Project Name	**Recallit**
Document Type	**SDLC Selection & Use Case Analysis**
Selected Model	**Agile Model**
Project Size	**Medium**
Team	**23524202014 - Ajwad Mohtasim**
	23524202052 - Nusair Ahmed Chowdhury
	23524202138 - Sumaiya Binta Shams

Contents

Problem Analysis

Recallit is a flashcard-based learning system designed to improve long-term memorisation through **active recall** and **spaced repetition**. Users select a study deck — for example, *C++ Advanced* — and work through cards one at a time. Each card initially hides its answer, prompting the user to recall the information independently before it is revealed. After viewing the answer, the user rates their performance using one of four options: **Again, Hard, Good,** or **Easy**.

The system employs the **Free Spaced Repetition Scheduler (FSRS)** to determine optimal review intervals. FSRS computes the next review date using three core metrics: *retrievability*, *stability*, and *difficulty*, ensuring that each card resurfaces at precisely the right moment to maximise retention with minimum effort.

Core Features

- User registration and login
- Deck creation and selection
- Adding, editing, and deleting flashcards
- Starting and managing review sessions
- Revealing flashcard answers on demand
- Rating recall difficulty per card
- Scheduling next review using the FSRS algorithm
- Tracking learning progress and performance statistics

Because Recallit combines both algorithmic scheduling and continuous user interaction, it must be developed using a model that supports flexibility, frequent testing, and iterative feedback.

Software Development Life Cycle (SDLC)

The SDLC for Recallit follows six structured stages, each building upon the previous to ensure a coherent and maintainable final product.

2.1. Planning and Requirement Analysis

The problem of inefficient memorisation is studied and the scope of Recallit is defined as a learning support system built on flashcard-based active recall and automated

spaced repetition scheduling.

2.2. Defining Requirements

System requirements are formally identified, covering: user authentication, deck management, flashcard CRUD operations, review session control, difficulty feedback collection, FSRS-based scheduling, and progress analytics.

2.3. Designing the Product Architecture

The system is partitioned into seven modular components:

Module	Responsibility
User Management	Authentication, registration, and profile data
Deck Management	Deck creation, selection, and metadata
Flashcard Management	Card creation, editing, and deletion
Review Session	Session flow, card display, and user interaction
FSRS Scheduling	Algorithmic interval calculation
Progress Tracking	Statistics, streaks, and learning analytics
Database	Persistent storage and data integrity

2.4. Building or Developing the Product

The system is implemented iteratively: the user interface is built first, followed by deck and flashcard operations, the review flow, and finally the scheduling logic.

2.5. Testing the Product

Testing verifies that questions display correctly, answers reveal as expected, difficulty ratings are stored accurately, and review dates are calculated by FSRS without error.

2.6. Deployment and Maintenance

Following deployment, the system is maintained through bug fixes, feature enhancements, and usability refinements guided by ongoing user feedback.

SDLC Model Selection

3.1. Models Under Consideration

Six widely-used SDLC models were evaluated for suitability:

#	Model	Key Characteristic
1	Waterfall	Sequential, phase-locked; changes are costly once a phase closes
2	V-Model	Pairs each development phase with a corresponding test phase
3	Iterative	Repeats a fixed cycle to progressively refine a partial solution
4	Incremental	Delivers the product in functional sub-units over successive releases
5	Spiral	Risk-driven; combines iterative development with systematic risk analysis
6	Agile	Collaborative, sprint-based; embraces change and continuous feedback

3.2. Project Characteristics

Factor	Assessment
Project size	**Medium**
Requirement clarity	**Partly clear, partly subject to change**
Risk level	**Moderate**
Time constraints	**Moderate**
Budget	**Moderate**
Customer involvement	**High**
Need for flexibility	**High**
Need for feedback	**High**

3.3. Selection Methodology

The most appropriate model was identified using the following five-step process:

1. **Analyse the Problem** — Understand Recallit's purpose: improving memorisation through active recall and spaced repetition.
2. **Evaluate Project Characteristics** — Confirm that the core concept is fixed but that the UI, review flow, analytics, and future features may evolve.
3. **Compare SDLC Models** — Assess each model against the criteria of flexibility, testing support, user feedback, and adaptability to change.
4. **Select the Suitable Model** — Choose the model with the highest weighted score that best fits the project's profile.
5. **Justify the Selection** — Articulate why the chosen model meets Recallit's specific development needs.

Model Comparison Matrices

*Scores: **1** = Poor fit **2** = Moderate fit **3** = Strong fit. Weighted score = Priority × Score.*

4.1. Raw Score Matrix

Criteria	Priority	Waterfall	V-Model	Iterative	Incremental	Spiral	Agile
Requirement clarity	2	3	3	2	2	2	2
Changing requirements	3	1	1	3	3	3	3
User feedback	3	1	1	3	3	2	3
Early delivery	2	1	1	2	3	2	3
Testing support	3	2	3	2	2	2	3
Risk handling	2	1	2	2	2	3	2
Flexibility	3	1	1	3	3	2	3
Suitability (medium)	2	2	2	3	3	2	3

4.2. Weighted Score Matrix

Criteria	Priority	Waterfall	V-Model	Iterative	Incremental	Spiral	Agile
Requirement clarity	2	6	6	4	4	4	4
Changing requirements	3	3	3	9	9	9	9
User feedback	3	3	3	9	9	6	9
Early delivery	2	2	2	4	6	4	6
Testing support	3	6	9	6	6	6	9
Risk handling	2	2	4	4	4	6	4
Flexibility	3	3	3	9	9	6	9
Suitability (medium)	2	4	4	6	6	4	6
Total		**29**	**34**	**51**	**53**	**45**	**56**

Selected SDLC Model and Justification

Selected Model: Agile Model

Weighted Score: **56** / **60** — Highest among all candidates

The **Agile model** is the most appropriate SDLC for Recallit for the following reasons:

- **Flexibility:** While the core concept of Recallit is well-defined, several components — the user interface, review flow, analytics dashboard, and person-

alisation features — are expected to evolve throughout development. Agile accommodates this through short, adjustable sprints.

- **Continuous feedback:** Agile integrates regular user and stakeholder feedback at the end of each sprint, which is essential for a user-facing learning application where usability directly affects learning outcomes.
- **Iterative releases:** Agile supports delivering an early working version of Recallit (e.g., basic flashcard review) and incrementally adding features such as FSRS scheduling and progress analytics.
- **Testing support:** Agile incorporates testing within every sprint cycle, enabling early detection of bugs in both the user interface and the scheduling logic.
- **Changing requirements:** As an educational tool, Recallit may evolve based on user behaviour and learning research. Agile is inherently designed to handle such evolving requirements without incurring significant rework costs.

Use Case Diagram

6.1. Methodology

The use case diagram was prepared using the following steps:

1. Define the scope of the Recallit system.
2. Identify all actors that interact with the system.
3. Enumerate all possible use cases within the system boundary.
4. Establish associations between actors and use cases, and include/extend relationships between use cases.
5. Render the diagram using standard UML notation.

6.2. Actors

- **User** — The primary actor. Interacts with the system to register, log in, manage decks and flashcards, conduct review sessions, and monitor learning

progress.

- **FSRS Engine** — An external scheduling component that computes the optimal next-review interval for each flashcard based on the user's performance rating.

6.3. Diagram

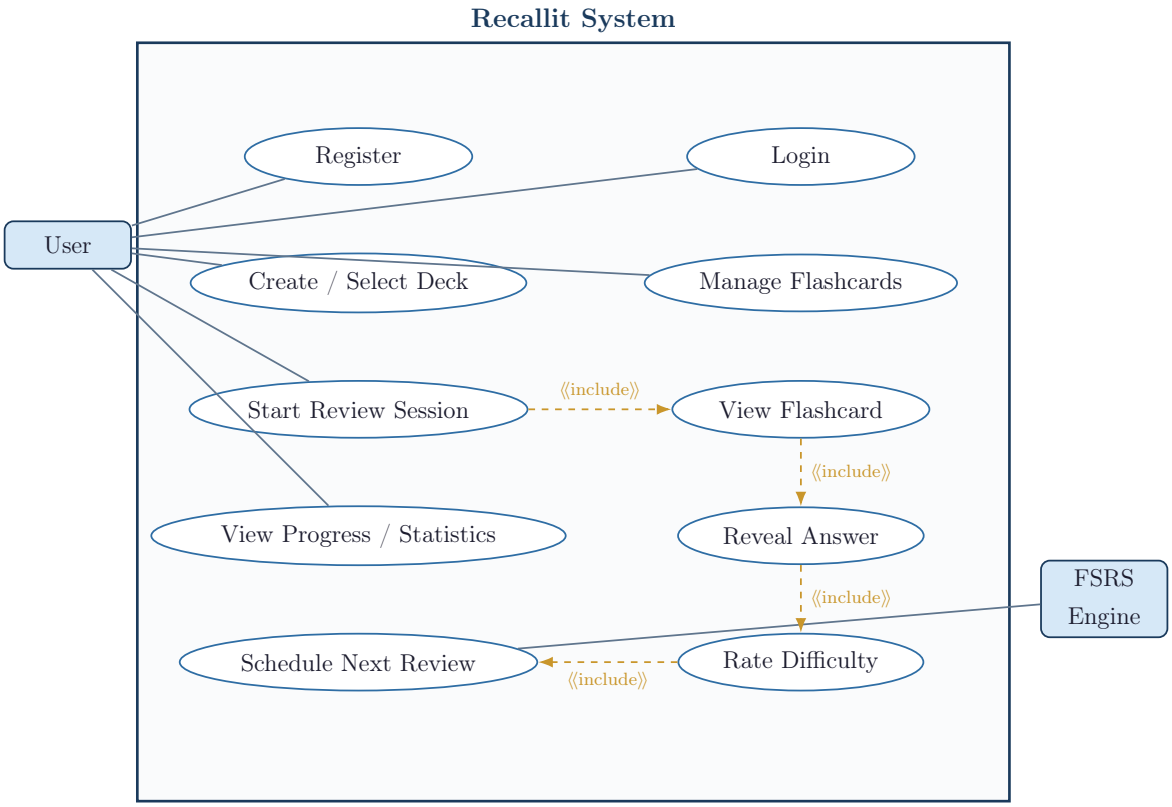

Figure 1: UML Use Case Diagram for the Recallit System

User Stories

#	Actor	User Story
1	**User**	As a user, I want to register an account so that I can access the system.
2	**User**	As a user, I want to log in so that I can view my decks and progress.
3	**User**	As a user, I want to create or select a deck so that I can study a chosen topic.
4	**User**	As a user, I want to add, edit, or delete flashcards so that I can manage my study materials.
5	**User**	As a user, I want to start a review session so that I can practise active recall.
6	**User**	As a user, I want to reveal the answer after thinking so that I can verify my recall.
7	**User**	As a user, I want to rate the difficulty of each card so that the system can schedule future reviews appropriately.
8	**FSRS Engine**	As the FSRS engine, I want to schedule the next review time so that cards resurface at the most effective interval.
9	**User**	As a user, I want to view my progress and statistics so that I can monitor my learning performance over time.

Conclusion

This report has presented a structured analysis of the SDLC model selection for the Recallit flashcard learning system. Through a weighted scoring evaluation of six candidate models, the **Agile model** emerged as the most suitable choice, achieving the highest total score of **56 out of 60**.

Agile's strengths in flexibility, iterative delivery, continuous testing, and regular user feedback align directly with the needs of a medium-sized educational application whose interface and features are expected to evolve during development.

The use case diagram provides a clear UML-compliant representation of the system's functional scope and actor interactions, while the user stories express the same requirements from an end-user perspective. Together, these artefacts form a complete and professional foundation for the Recallit software development process.